

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术
实验项目编号 2 实验名称 简单功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、基本思路说明、具体设计细节

1 simple cpu 的设计思路及具体实现

实验指导书里的错误示例是对每一个指令单独进行了分类讨论, 没有找指令的共性, 也没有根据指令的共性来决定各个信号控制, 这告诉我什么是错误的。

同时, 指导书中对 `ALUop` 巧妙的设计给了我思路的启发: 我们可以对 45 条指令简单分类, 根据其各自的类别, 结合指令自身的特点, 决定各个控制信号, 操作数等。

因此, 我的设计是, 对每个主要的部件进行探究, 结合指令的分类, 决定各个控制信号, 操作数。这样每个部件设计好了输入、输出, 最终的 CPU 也就设计好了。

总设计图在理论课的 PPT 里给的已经非常具体、详细了(除了没有专门给出移位器), 于是这里的总设计图借用这张图:

组装好的单周期处理器数据通路

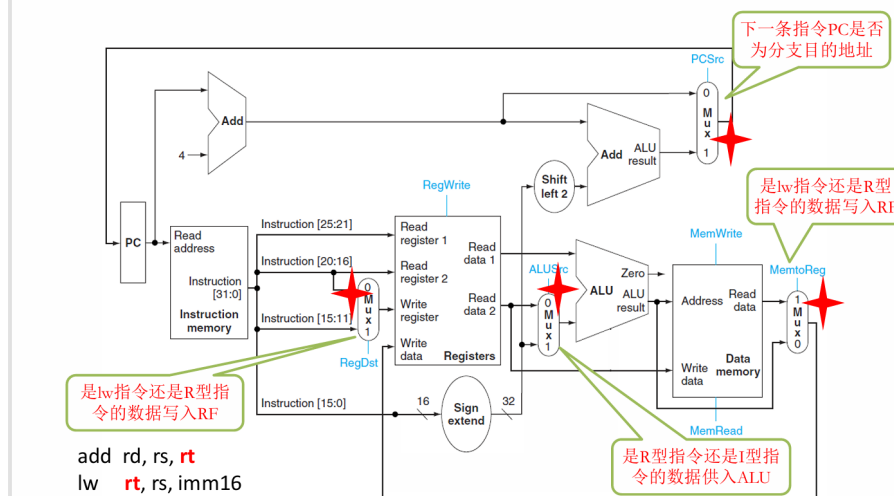


图 1: simple cpu 的总设计图(借用理论课 ppt)

具体设计思路和步骤如下:

- 将指令切片, 分成 opcode、rs、rt、rd、shamt、funct、imm 等部分(具体代码略); 以备后续直接使用(即下图的这个部分):

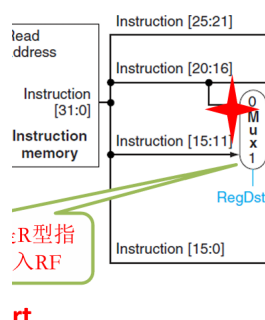


图 2: 对指令切片的示意图

• ALU 实例化

ALU 的实例化重点是操作数和 ALUOp 的选择。

首先, 对于 ALUOp, 我将指令分成了四类, 分别找 opcode/funct 和 ALUOp 的对应关系(体现了对规律的寻找和掌握), 具体如下:

- R 型运算指令类: opcode == 6'b000000 R 型指令的具体操作由低 6 位 'funct' 决定。通过观察 'funct' 的规律, 可以直接拼接出 ALUOp:

当 'funct[5] == 1'b0': 包含了 sll, srl, sra 和 movz, movn 等移位/条件传送。这些指令要么由专门的移位器处理, 要么不需要 ALU 做复杂运算, 因此统一输出 'ADD'。

当 'funct[3:2] == 2'b00', 对应加减运算, 此时 opcode='funct[1], 2'b10', 恰可以对应上正确的运算方式(这也体现了指令集设计的精巧)。

当 'funct[3:2] == 2'b01' 时, 对应逻辑运算, opcode='funct[1], 1'b0, funct[0]'

当 `funct[3:2] == 2'b10` 时, 是比较运算, `'opcode = funct[0], 2'b11'` (对 `funct[0]` 取非恰恰可以决定是 `slt` 还是 `sltu`, 使得设计很简洁)

- I 型算术逻辑指令类: `'opcode[5:3] == 3'b001'`

对应算术运算, 但是是立即数类。I 型指令的 `'opcode` 低 3 位和 R 型指令的 `'funct` 低 3 位高度对应, 这里的推导几乎完全复用了上面的逻辑, 只是把判断条件从 `'funct` 换成了 `'opcode` 的低 3 位:

加法: `'opcode[2:1] == 2'b00` -> 结果同为 `'010` 逻辑: `'opcode[2] == 1'b1` -> 结果同上提取 `'opcode` 对应位比较: `'opcode[2:1] == 2'b01` -> 生成 `'111` 或 `'011`

- 分支比较类: `'(opcode[5:2] == 4'b0001) || (opcode == 6'b000001)'`

对应 `'beq, bne, blez, bgtz` (0001xx) 和 `'bltz, bgez` (000001)。

分支指令需要判断两个寄存器之间的大小关系, 或者比较寄存器和 0 的关系。之前思考过是应该用 `slt` 还是 `SUB`。但是如果是 `slt`, 只能告诉是否小于。对于 `blez` (小于等于 0 则跳转) 是不好判定的 (因为 `slt` 给我们的是小于, 而如果这个数是 0 呢? 这个时候就判断不了了)。而如果我们使用 `SUB`, 如果相减结果是 0 (ALU 的 Zero 标志位拉高), 说明两数相等; 此时可以通过查看 ALU 减法结果的符号位来进一步判断大小关系。因此全部赋值为 `SUB`。

- 访存类: `'opcode[5:4] == 2'b10`

对应所有的 Load/Store 指令 (opcode 都是 `'10xxxx`)。

访存指令计算内存地址的公式是需要做加法计算地址, 所以全部赋予 `ADD`。

因此, `AluOp` 的设计代码是:

```
1 assign ALUOp = (opcode == 6'b000000)? ((funct[5] == 1'b0)? `ADD : //只要最高位是0 (包含movz,
    movn, shift), 交给ADD处理, 阻止其进入算术比较切片
2 (funct[3:2] == 2'b00)? {funct[1], 2'b10}:
3 (funct[3:2] == 2'b01)? {funct[1], 1'b0, funct[0]}:
4 (funct[3:2] == 2'b10)? {~funct[0], 2'b11}:
5 `SUB) : //算术类1
6 (opcode[5:3] == 3'b001)? ((opcode[2:1] == 2'b00)? {opcode[1], 2'b10}:
7 (opcode[2] == 1'b1)? {opcode[1], 1'b0, opcode[0]}:
8 (opcode[2:1] == 2'b01)? {~opcode[0], 2'b11}: //修改: sltiu
9 `SUB) : //算术类2
10 ((opcode[5:2] == 4'b0001) || (opcode == 6'b000001))? `SUB:
    //比较分支; 用sub可以看符号位是0还是1, 以及通过zero判断是不是0
11 ((opcode[5:4] == 2'b10)? `ADD : //访存类
12
13 `SUB;
```

接下来, 对操作数 A 与 B 进行讨论:

操作数 A 永远是通用寄存器 `rs` 读出的数据, 十分简单。

操作数 B 可以来自寄存器 `rt`, 也可以来自立即数 `imm`, 还可以强制赋 0。这里将操作数 B 分成了四类:

- B 被强制为 0: 因为一些指令需要让操作数 A 和 0 进行比较, 那么操作数 B 只能强制赋值是 0 了。(后续查看 ALU 计算的结果的最高位和 Zero 的值, 就可以得出比较结果了)

比如: `'opcode[5:1] == 5'b00011`: 对应 `'blez(000110), bgtz(000111)`。

比如: `'opcode == 6'b000001`: 对应 `'bltz, bgez`。

比如 `'opcode == 000000 && funct[5:1] == 00101`: 对应 `'movz(001010), movn(001011)`。

- B 被赋予符号扩展立即数 (Sign-Extended Imm)

主要是: 算术型立即数指令和所有的访存地址偏移量计算, 立即数都必须进行带符号扩展, 即把 16 位立即数的最高位 (符号位) 复制到高 16 位, 以保证负数依然是负数。

比如: 'opcode == 6'b001001': 'addiu'

'opcode[5:1] == 5'b00101': 'slti(001010), sltiu(001011)'

'opcode[5:3] == 3'b100': 所有的访存取指令 (如 'lb, lh, lw, lbu, lhu' 等 100xxx)

'opcode[5:2] == 4'b1010': Store 指令 (如 'sb, sh, sw' 等 1010xx)

- B 被赋予零扩展立即数

当指令是逻辑型立即数指令, 立即数只能进行无符号扩展, 即高 16 位强制补 0。因为逻辑运算改变符号位是没有意义甚至会导致错误的。

比如说: 'opcode[5:1] == 5'b00110': 对应 'andi(001100), ori(001101)'

'opcode == 6'b001110': 对应 'xori(001110)'。

- B 被赋予寄存器数据 (默认情况): 'rdata2'

即代码:

```
1 assign B = ((opcode[5:1] == 5'b00011) || (opcode == 6'b000001) || ((opcode ==
2 6'b000000) && (funct[5:1] == 5'b00101))) ? 32'b0: //bgez等和0跳转的;以及movz 和
movn
3 ((opcode == 6'b001001) || (opcode[5:1] ==
4 5'b00101) || (opcode[5:3] == 3'b100) || (opcode[5:2] == 4'b1010) || (opcode ==
6'b101110)) ? extend_imm: //算术里面立即数符号扩展/访存里的
((opcode == 6'b001110) || (opcode[5:1] == 5'b00110)) ? { 16'b0, imm}:
//算术里面立即数逻辑扩展
rdata2; //一般来说, R指令
```

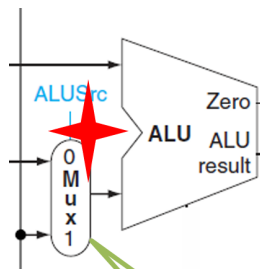


图 3: ALU 实例化

然后最后实例化 ALU 即可。

• 移位器 Shifter 的实例化

在 MIPS 指令集中, 一共只有 6 条标准移位指令, 且全都是 R 型指令 (opcode = 000000)。它们的 funct 编码很有规律: 高位 3 位都是 0; 并且根据 funct 对应的后两位, 设计 Shifter 对应的 Shift op, 从而可以直接截取 funct[1:0] 来得到对应的 Shiftop。

shift_A 表示被移位的数, 直接来源于寄存器 rt

shift_B 表示移位多少位。sll, srl, sra 对应着立即数 shamt 这么多位的移位, 他们的共同特点是 funct[5:2] == 4'b0000, 此时对 shamt 做零扩展即可; 而其他的情况的移位来自于寄存器 rs 的数据。

故代码如下:

```

1 assign Shiftop = ((opcode == 6'b000000) && (funct[5:3] == 3'b000)) ? funct[1:0] :
2   `nothing; //普通的shift, 其他的直接拼接即可
3 assign shift_A = rdata2;
4 assign shift_B = (funct[5:2] == 4'b0000) ? {27'b0, shamt} : //ptt:{27'b0, shamt}
5   rdata1; //要么来自于立即数, 要么直接来源于rs

```

• register 寄存器的设计

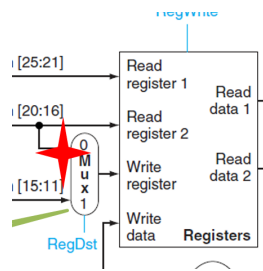


图 4: 寄存器的设计

实例化后是:

```

1 reg_file Reg_file(.clk(clk), .waddr(RF_waddr), .raddr1(reg_raddr1), .raddr2(reg_raddr2)
2   , .wen(reg_wen), .rdata1(rdata1), .rdata2(rdata2), .wdata(wdata));

```

首先, 我们看控制信号:

1. reg_wen, 写使能: 有以下几种情况被赋值为 1:

第一类: 普通的 R 型指令 (算术、逻辑、移位等), 但是剔除了 jr/movz/movn 即:

```

1 ((opcode == 6'b000000) && ((funct != 6'b001000) && (funct[5:1] != 5'b00101)))

```

第二类: 条件传送指令 (movn 和 movz)

这个只看 rt 之中的数据是否等于 0 就行; 而 movn 和 movz 的差异是 funct 的第 0 位, 那么只需要先确定是 Movn/movz, 而后将 funct 的第 0 位, 和 rt 与 0 的比较结果进行异或, 就可以直接涵盖两种情况.

```

1 ((opcode == 6'b000000) && (funct[5:1] == 5'b00101) && ((rdata2 == 0) ^ funct[0])) ||
   //movn movz

```

第三类是 ((opcode[5:3] == 3'b001) || (opcode[5:3] == 3'b100)), 对应着算术 I 型指令, 和 Load 类指令

第四类是 jal: (opcode == 6'b000011)

2. 第二个重点是读地址的选择, 直接是 rs 和 rt, 因为如果要读寄存器, 寄存器编号就是来自于 rs 和 rt, 没有其他可能性了。

3. 第三个重点是写地址:

如果是立即数类, 那么是写入 rt; 如果是一些算术类、寄存器跳转并链接、movn 和 movz 等, 就是写入 rd; jal 是默认写入 31 号 (因为 jal 会自动将下一条指令的地址存入寄存器 31) 中。

4. 最困难的是 wdata——写入数据。因为会涉及字、半字、字节的 Load, 甚至不对齐的写入。

具体代码如下: 对于取出来的半字、字节, 分别需要进行扩展(符号位扩展 or 零扩展); 对于 jal 和 jalr, 需要将下一条指令的地址存进去; 对于移位类计算, 将 shift_result 存进去即可; 对于 lui, 就将立即数下方填充 16 个 0(相当于向左移位); 其他的就填入默认的 ALU 的结果即可。

```
1 assign wdata = (opcode == 6'b100000)? {{24{byte_data[7]}},byte_data}: //lb
2 (opcode == 6'b100100)? {24'b0,byte_data}: //lbu
3 (opcode == 6'b100001)? {{16{doubytes_data[15]}},doubytes_data}: //lh
4 (opcode == 6'b100101)? {16'b0,doubytes_data}: //lhu
5 (opcode == 6'b100011)? mrddata1: //lw
6 ((opcode == 6'b100010) || (opcode == 6'b100110))? lwr1: //lwr /lwl
7 ((opcode == 6'b000011) || (opcode == 6'b000000 && funct == 6'b001001))? easy_PC+4: //
   jal
8 ((opcode == 6'b000000)&&(funct[5:3]== 3'b000))? shift_result: //移位的
9 ((opcode == 6'b001111))? {imm,16'b0}: //ptt:左移会有数值溢出
10 Result;
```

关键: 如何取出对应的半字、字节、字, 甚至是非对齐的数据?

对于字, 直接选择取出的 mrddata1 就行。

对于半字, 根据地址末位是 00 还是 10, 分别对应选择字的前半段还是后半段。(因为是小端序, 所以低地址存低字节, 00 是低地址, 所以是低字节: mrddata1[15:0]; 10 是高地址, 所以是高字节, mrddata1[31:16])

对于字节, 其实也是一样的, 地址末位是 00 那么就是最低的字节, 即 [7:0], 其他的以此类推, 略。

对于非对齐的 Load: lwr lwl, 相对来说很难理解!

总结下来: lwr 指的是填充寄存器的右侧, lwl 指的是填充寄存器的左侧。

寄存器的左侧 = 高位且小端序高地址 = 高位, 所以 lwl 指向非对齐数据的高(最大)地址, 并提取该对齐字内向下到偏移 0 的所有内容, 并拼到寄存器的左侧。

因为寄存器的右侧 = 低位且小端序低地址 = 低位, 所以 lwr 指向非对齐数据的低(最小)地址, 并提取该对齐字内向上到偏移 3 的所有内容, 并拼到寄存器的右侧。

因此, 我们制定掩码, 根据掩码, 我们选择这个字节, 是 memory 中的内容, 还是 register 中的内容:

比如说, lwl 中 mraddr 是 10, 那么我们应该提取 10 向左到偏移 0 的所有内容, 并拼到寄存器的左侧, 所以掩码应当是 1110; 或者比如说 lwr 的 mraddr 是 10, 那么我们应该提取 10 向上到偏移 3 的所有内容, 并且拼到寄存器的右侧, 所以掩码应该是 0011。具体如下:

```
1 assign lyanma = (mraddr1[1:0] == 2'b00)? 4'b1000:
2 (mraddr1[1:0] == 2'b01)? 4'b1100:
3 (mraddr1[1:0] == 2'b10)? 4'b1110: //lwl的掩码
4 4'b1111;
5
6 assign ryanma = (mraddr1[1:0] == 2'b00)? 4'b1111:
7 (mraddr1[1:0] == 2'b01)? 4'b0111:
8 (mraddr1[1:0] == 2'b10)? 4'b0011:
9 4'b0001; //lwr的掩码
10
11 assign yanma = (opcode[2]==0)? lyanma: ryanma; //掩码
```

而后, 我们准备 memory 中的数据: 因为我们需要根据掩码来选择是用 memory 中的数据, 还是 register 的数据, 所以我们需要把 memory 中的数据和 register 中的数据对齐。比如说是 lwl 的时候, mraddr 是 01,

说明要填充寄存器的左侧(高位),而 01 就是内存中的高位地址,一直到 0 都是我们所需要的,那么就需要将这 [15:0] 的数据移动到寄存器的左侧,即移动 16 位。比如说是 lwr 的时候,mraddr 是 01,说明需要填充寄存器的右侧,而 01 就是内存中的低位地址,一直到 3 都是我们需要的,那么就将 [31:8] 向右移动 8 位,到达寄存器的右侧 [23:0]。具体如下。

```

1      assign shifted_mem_data =
2          (opcode == 6'b100010) ? // lwl
3          ( (mraddr1[1:0] == 2'b00)? mrddata1 << 24 :
4          (mraddr1[1:0] == 2'b01)? mrddata1 << 16 :
5          (mraddr1[1:0] == 2'b10)? mrddata1 << 8 :
6          mrddata1 ) :
7          (opcode == 6'b100110) ? // lwr
8          ( (mraddr1[1:0] == 2'b00)? mrddata1 :
9          (mraddr1[1:0] == 2'b01)? mrddata1 >> 8 :
10         (mraddr1[1:0] == 2'b10)? mrddata1 >> 16 :
11         mrddata1 >> 24 ) :
12         mrddata1;

```

最后根据掩码,来选择寄存器中写入的内容,是寄存器原来的内容,还是经过移动的内存内容:

```

1      assign lwrl[7:0] = yanna[0] ? shifted_mem_data[7:0] : rdata2[7:0];
2      assign lwrl[15:8] = yanna[1] ? shifted_mem_data[15:8] : rdata2[15:8];
3      assign lwrl[23:16] = yanna[2] ? shifted_mem_data[23:16] : rdata2[23:16];
4      assign lwrl[31:24] = yanna[3] ? shifted_mem_data[31:24] : rdata2[31:24];

```

• memory 内存的接口设计

重点是以下几个: 1. Address, 因为我们一次只能取出一个字(32 位),但是是按字节编址(8 位)的,所以 CPU 直接把地址的高 30 位送给内存控制器,而不管最低两位,从而取出一个字。这时,系统规定,地址的后两位必须是 00,如果地址的后两位不是 00,那么就会抛出一个地址对齐异常。

```

1      assign Address = {Result[31:2], 2'b00} ;

```

2. 允许写入,只要是 store 类型的,就允许写入:

```

1      assign mwren = (opcode[5:3] == 3'b101)? 1: 0

```

3. 接下来是 mstrb: 掩码,如果是 1 表示这个字节写的是 mwdata 上的,如果是 0 表示这个字节是原来的,不可以写。

如果是字 store 的,就应该全部写入,1111;如果是半字,如果是 00 那么写入低地址,0011,10 写入高地址,1100;如果是一个字节,那么如果是 00 就写入最低字节,以此类推,所以用了一个队 0001 的位移,第几个字节就移位几次。

最难的又是非对齐 store....。其中,swl 是寄存器的左边数据,是高位,应该放到大地址之中。所以就应该放到这个字的低地址中(因为上一个字放不下,所以只能放在这里),所以提供的地址就是这个数据应该存到的最高的地址,一直到 0 都应该是被存入的数据。所以比如说 swl 下,mwaddr[1:0] 是 10,所以应该存的是 00 01 10 这三个位置,所以掩码是 0111.swr 反着来即可。

```

1      assign mwstrb = (opcode == 6'b101011)? 4'b1111: //sw
2      (opcode == 6'b101001)? ((mwaddr[1:0] == 2'b00)? 4'b0011:4'b1100): //sh

```



```

3      (opcode == 6'b101000)? (4'b0001 << mwaddr[1:0])://sb, 根据地址最后两位进行移位
4      (opcode == 6'b101010) ? ( // swl(?)
5      (mwaddr[1:0] == 2'b00) ? 4'b0001 :
6      (mwaddr[1:0] == 2'b01) ? 4'b0011 :
7      (mwaddr[1:0] == 2'b10) ? 4'b0111 : 4'b1111
8      ) :
9      (opcode == 6'b101110) ? ( // swr
10     (mwaddr[1:0] == 2'b00) ? 4'b1111 :
11     (mwaddr[1:0] == 2'b01) ? 4'b1110 :
12     (mwaddr[1:0] == 2'b10) ? 4'b1100 : 4'b1000
13     ) :
14     4'b1111;

```

4. 是 mwdata: 我们考虑将需要写入的数据复制, 填满整个 mwdata, 这样直接供掩码进行选择, 这样选择出来的 rdata 就是我们所需要的。如果是字, 那么直接就是寄存器中的 rdata2; 如果是半字, 就把这个寄存器中的 rdata2 的半字(低半字)复制两次; 如果是字节, 就把这个寄存器中的 rdata2 的字节(低字节)复制四次:

最困难的还是非对齐.... 比如 swl 要存的是寄存器的左侧的数据; 如果地址是 01, 那么要将这个数据存到内存里的 00, 01 的位置, 那么就要取 2 个字节, 就要将寄存器的最高两字节放入 mwdata 里面:rdata2[31:16]。其他同理。比如 swr 要存的是寄存器右侧的数据, 低位, 如果地址是 01, 那么要将这个数据存到从 01 到 11 的位置, 一共 3 个字节, 所以是 rdata2[23:0] 这三个字节。

```

1      assign mwdata = (opcode == 6'b101110)?( //swr
2      (num == 2'b01)? {rdata2[23:0], 8'b0}: //从最低位之上开始存,
3      (num == 2'b10)? {rdata2[15:0], 16'b0}:
4      (num == 2'b11)? {rdata2[7:0], 24'b0}:
5      rdata2
6      ):
7      (opcode == 6'b101010)? ( //swl
8      (num == 2'b00)? {24'b0, rdata2[31:24]}:
9      (num == 2'b01)? {16'b0, rdata2[31:16]}:
10     (num == 2'b10)? {8'b0, rdata2[31:8]}:
11     rdata2
12     ):
13     (opcode == 6'b101000) ? {4{rdata2[7:0]}} : // sb: 将最低字节(ptt!)复制 4 份
14     (opcode == 6'b101001) ? {2{rdata2[15:0]}} : // sh: 将低半字复制 2 份
15     rdata2 ; //写入数据的时候, 按照小端序, 进行看

```

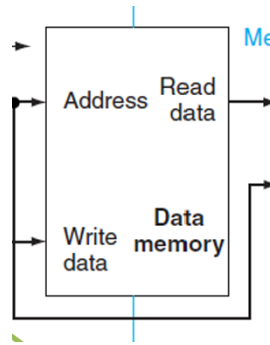



图 5: 内存 memory 的图示

• PC

PC 是一个寄存器,每一次时钟沿到来时,如果 rst 的那么就置零,否则就置 next_PC。

关键是要算 next_PC,有四个来源,分别如下(有来自于 PC+4,有来源于 PC+4+ 立即数左移,有来自于寄存器的,有 easy_PC[31:28], target_address, 2'b00, 选择如下:

```

1      assign easy_PC = PC + 4;
2      assign diff_PC = {extend_imm[29:0], 2'b00} + easy_PC;
3      assign next_PC = (opcode[5:1] == 5'b00001)? {easy_PC[31:28], target_address, 2'b00}:
           //j和jal
4      ((opcode == 6'b000000)&&(funct[5:1] == 5'b00100))? rdata1: //jr和jalr
5      (PC_src == 1)? diff_PC :
6      easy_PC;

```

关键是,如何选择什么时候是 diff_PC 呢,什么时候是 PC+4 呢?

主要是一些 branch 指令,如果满足对应的条件,next_PC 就是 diff_PC。

其中:beq 和 bne 看的是 ALU 计算出来的是不是 0,那么将 zero 和 beq/bne 不同的那一位 (0 位) 异或起来就可以得出结果!

blez 和 bgtz 看的是是否小于等于 0,那么看 Result[31](小于)、Zero(等于),然后和他们不一样的那一位进行异或,也可以直接得到结果。

最后是 ((opcode == 6'b000001) 是 REGIMM 的,这个时候问的是是否结果小于 0,如果小于 0 就是 Result[31],而且异或的是 BLTZ 和 BGEZ 的不同的那一位 rt[0],从而得出结果。

```

1      assign PC_src = ((opcode[5:1] == 5'b00010 ) && (Zero^opcode[0]))? 1: //beq和bne
2      ((opcode[5:1] == 5'b00011 ) && (opcode[0]^((Result[31])||Zero)))? 1: //blez和bgtz
3      ((opcode == 6'b000001) && ((rt[0]) ^ (Result[31]))) ? 1 : //
4      0;

```

1 其他底层建构的修改

1. 为了无符号比较,加入 sltu,用是否 $A < B$ 来进行无符号数直接的比较(二中有提到我的错误设计)
2. 为了实现移位,加入模块 shifter,根据其对应的 opcode 的特定位来设计对应的 Shiftop 控制信号,然后实现算术和逻辑移位(其中有个很隐藏的 bug,二中将详细介绍)

二、 实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug, 逻辑仿真和 FPGA 调试过程中的难点等)

我们可以看到, 关键点是: 如何区分出这一类指令(唯一又简短), 这需要对这一类指令的特征进行提取; 如何设计出对应的信号, 是可以根据指令的特征进行异或/片段提取直接得到控制信号。

现在指出遇见的一些 bug 和问题思考。

1. 如何理解非对齐访存?

最开始一直无法理解, 后来想到: 这个是为了将一个字存到没有对齐的(指低位不是 00)的位置上。那么比如: `swr` 是为了存寄存器的右侧数据, 低位。低位应该是存在内存的高位, 这是因为低位存不下了, 所以只能在高位截断, 所以提供的地址是存入内存的最低地址。比如: `lwl` 是为了将内存的数据放入寄存器的左侧, 高位。这应当是这个数据也是非对齐的, 这个数据的低位在前一个字的高位, 高位在下一个字的低位, 所以提供的地址是数据的最高地址。

绘图如下:

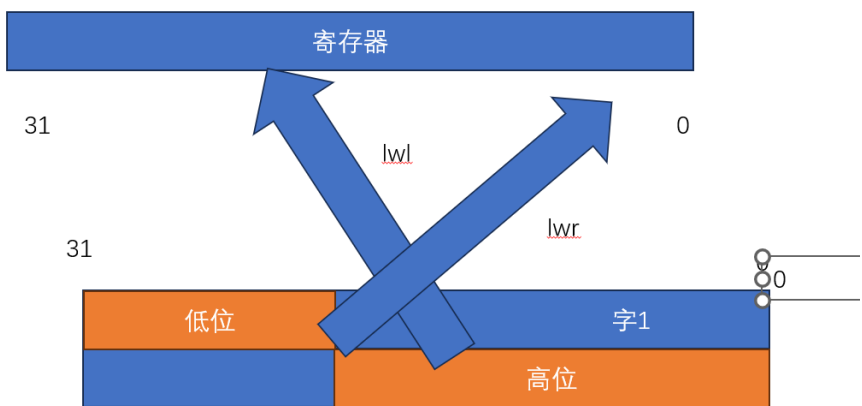


图 6: load 的图示

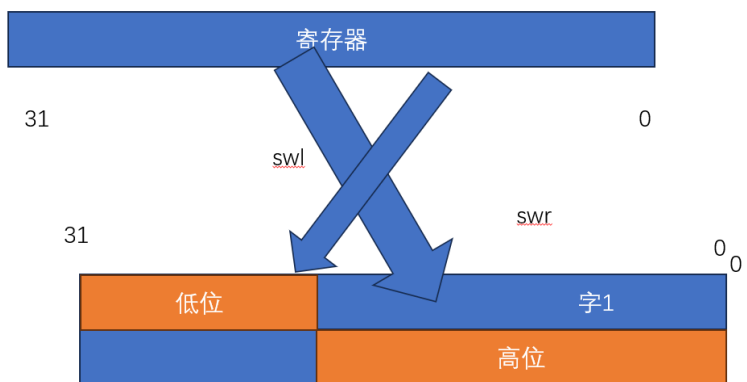


图 7: swl 和 swr 的图示

2. sltu 的错误

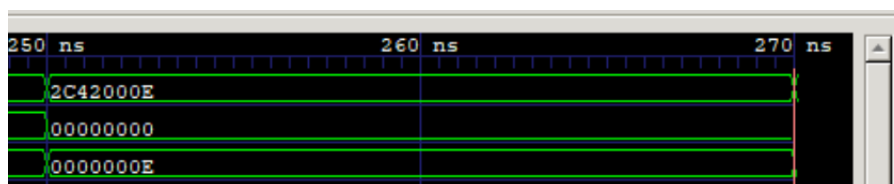


图 8: 错误波形

明明操作数是 2 和 14,却最终是

```
1 Yours: RF_waddr = 02, (RF_wdata & 0xffffffff) = 0x00000000
2 Reference: RF_waddr = 02, (RF_wdata & 0xffffffff) = 0x00000001
```

说明肯定是 sltu 的逻辑错了,2 本来小于 14 应该是 1 来着,结果我给的是 0。

回到 alu 初始逻辑:

当时判断 Result 的时候有一步:

```
1 (ALUop == `SLTU)? {31'b0,~CarryOut}:
```

看似通过是否有借位判断是没错的,然而我当时用的是 33 位的加法器,这样可以更好判断溢出;然而这就导致我的 carryout 实际上是我符号扩展的 A 和 B 的 carryout,而非本来的无符号数(无符号数的比较应该进行零扩展),所以出现了错误。这属于是之前的架构直接套用,就会出现想当然的问题。改正:

```
1 (ALUop == `SLTU)? {31'b0,(A<B)}:
```

3. 地址的错误截断问题

问题排查到

```
1 Reference (标准地址): 0x00003ffc (二进制: 0011 1111 1111 1100)
2 Yours (我的地址): 0x00000ffc (二进制: 0000 1111 1111 1100)
```

说明地址的高位可能被错误截断了。后来想起来,因为在 ideal_mem.v 中看见 parameter ADDR_WIDTH = 14, 在 simple_cpu_top.v 里面我看实例化的 addr 也是 14 位,所以所有的地址相关的我都设置的是 14 位。

但是确实错的。后来才知道,实例化 14 位是因为这里测试只需要 14 位,但是实际上我应该设计 32 位的地址,我们可以看到,simple_cpu 的实例化的接口是:

```
1 output [31:0] Address
```

需要 32 位,所以不管别人如何实例化,我应该坚持本来设计的接口原则,不得舍近求远。

4. 因为字是一整个取出,但是是按字节编址,所以地址应该将低两位设成 0.

```
1 ERROR: at 5070ns. Yours: Address = 0x00000212, Write_strb = 0xc, (Write_data
2 & 0xffff0000) = 0xfff70000 Reference: Address = 0x00000210, Write_strb = 0xc,
3 (Write_data & 0xffff0000) = 0xfff70000opcode是有这个SH
```

发现低两位应该是 0,但是我是 2,查询资料后得出,应该设成 0.

5. 算术移位并没有实现, 仍然是逻辑移位(隐蔽的)

```
1 ERROR: at 2430ns.
2 Yours: RF_waddr = 04, (RF_wdata & 0xffffffff) = 0x04c3b2a1
3 Reference: RF_waddr = 04, (RF_wdata & 0xffffffff) = 0xfcc3b2a1
```

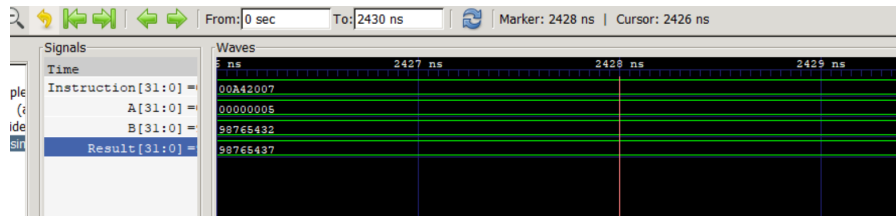


图 9: shifter 错误关键波形图

这个时候应该是算术右移, 但是如果把 0x98765432 进行逻辑右移(高位补 0)移 5 位, 结果刚好就是我的 0x04c3b2a1, 如果把 0x98765432 进行算术右移(高位补符号位 1)移 5 位, 结果刚好就是标准的 0xfcc3b2a1。说明进行的是逻辑右移。但是我明明写的是:

```
1 (funct[1:0] == 2'b11) ? ((B) >>> shift_amount)
```

这难道不是一个算术移位吗? 甚至

```
1 (funct[1:0] == 2'b11) ? ($signed(B) >>> shift_amount)
```

仍然会出现相同的问题。

查询资料后得出: Verilog 在计算表达式时, 上下文环境会向内部传递。Result 的整个三目运算符表达式被定性为“无符号环境”, 那么在这个环境里的所有操作数, 都会被当成无符号数处理。»> 是算术右移。如果左侧操作数是有符号数, 它移位时就补符号位(最高位保持不变); 如果左侧操作数是无符号数, 它移位时就补 0(退化成了逻辑右移 »)。虽然我写了 \$signed(B), 但在外层“无符号环境”的强力压制下, \$signed(B) 会在参与运算前被剥夺有符号属性, 重新当作无符号数。所以: »> 发现自己的左边变成了无符号数, 于是它就老老实实地去补 0 了, 导致想要的算术右移变成了逻辑右移。

因此, 我将 »> 给特别算出来, 然后再进行选择:

```
1 wire signed[31:0] signed_A = A; // 强制赋予符号
2 wire signed [31:0] signed_res = signed_A >>> B[4:0]; // 纯 signed 环境下的算术右移
3
4 // 3. 最后再用多路选择器进行分配
5 assign Result = (Shiftop == `left_shift) ? sll_res :
6 (Shiftop == `ari_right) ? signed_res : // 此时 signed_res 已经是正确算出 0xfc...
7 // 的结果了
8 (Shiftop == `logic_right) ? srl_res :
9 A;
```

三、 对讲义中思考题(如有)的理解和回答

1. 什么是译码?

R-Type 指令	func (6-bit)	ALUOp (3-bit)	ALUOp编码
add	10 00 00		
addu	10 00 01	010	ADD/SUB: func[3:2] == 2'b00
sub	10 00 10		
subu	10 00 11	110	ALUOp = {func[1], 2'b10}
and	10 01 00	000	
or	10 01 01	001	逻辑运算: func[3:2] == 2'b01
xor	10 01 10	100	ALUOp = {func[1], 1'b0, func[0]}
nor	10 01 11	101	

图 10: ALUOp 的规律寻找 (来自于指导书)

是将 32-bit 二进制指令串,分段切开,然后根据指令类别进行分类,转化成指令功能所需的控制信号 + 操作数。

2. 什么是控制信号?

是部件的各个输入端口的数据选择,还有操作类型,以及寄存器/数据内存读写控制等信号

3. 什么是操作数?

指令中的立即数、寄存器中的数据、数据内存访问的地址与数据等

4. 如何生成控制信号?

通过指令类型,找到不同指令间的共性特征,进行分类,然后寻找指令和控制信号之间的关联,生成对应的控制信号。

5. ALUOp 的编码有什么规律?

答:与 func 的切片十分相关:

但是还可以进一步优化:

```
1 assign ALUOp = {func[1], ~func[2], func[2] & func[0]};
2 //
   第一个都是func[1],第二个是相反的1和0, 然后发现恰和func[2]是相反的; 第三个如果func[2]是0的算术运算那么就
```

6. 状态机第三段处理输出逻辑: 用 current_state 作为判断条件或赋值变量, 尽量不使用 next_state (why? 报告里写出自己的思考)

1) 在设计 Mealy 型状态机时, 输出按照状态和当前输入决定下一步。而 next_output 本身就是由 current_state 和当前输入计算得来的。如果用 next_state 再去结合下一状态判断输出, 属于典型的逻辑重叠。

2) 加长了组合逻辑关键路径: current_state 是经过触发器(寄存器)打一拍后的输出信号, 信号在时钟沿到来后直接就绪, 路径极短。next_state 是第二段状态机(组合逻辑)的计算结果, 它本身已经包含了一段“输入信号 + 现态”的计算延迟。

如果在第三段使用 next_state 来判断输出, 硬件综合出来的电路就会把“次态计算组合逻辑”和“输出计算组合逻辑”串联在一起。这会大幅增加两条组合逻辑的级联延迟, 拉低整个系统能够运行的最高时钟频率。如果用 current state 的话, 输出计算和次态计算就可以并行了。

3)current_state 由于是由触发器直接输出的, 信号非常干净、稳定, 没有毛刺。next_state 作为一个纯组合逻辑信号, 在多个输入信号发生跳变时, 极易产生中间态的电压毛刺, 这些毛刺就会毫无保留地传递给下一级电路。

四、 补充进行实验:RISC-V 单周期 CPU 设计

4 设计理念转变:从“逐指令对比”到“指令分类”

在之前的 MIPS 设计中,我的做法是对每一条具体的指令进行码字对比。这种方式在指令数量增加时会导致代码极其臃肿且难以维护。在本次 RISC-V 设计中,我采取了更加系统化的分类处理法。

首先,我通过解析 `Instruction[6:0]` (opcode) 将指令划分为几个大的功能类:

- **计算型:**包括 R 型(寄存器-寄存器)和 I 型(寄存器-立即数)。
- **访存型:**分为 Load 指令(`is_L`)和 Store 指令(`is_S`)。
- **控制流:**包括条件分支(`is_B`)和无条件跳转(`is_J_JAL/JALR`)。
- **高位立即数:**LUI 与 AUIPC。

这种分类方式使得后续的立即数生成非常直观。由于 RISC-V 的立即数分布较为零散,我根据指令类别统一进行符号扩展,例如 B 型指令的立即数构造公式为:

$$\text{imm_B} = \{\{20\{Inst[31]\}\}, Inst[7], Inst[30:25], Inst[11:8], 1'b0\}$$

4 ALU 控制与多路选择逻辑

在 ALU 的操作控制上,我利用 RISC-V 严格的 `funct3` 定义进行映射。A 端操作数在 AUIPC 时选择 PC,其余通常选择寄存器值;B 端则在 R 型与 B 型指令下选择寄存器,其余选择立即数。这种逻辑极大简化了控制单元的复杂度,使得代码结构清晰。

五、 附加任务二:RISC-V 多周期 CPU 设计

5 多周期下的时序哲学与分支处理

多周期 CPU 的核心挑战在于状态机的划分。与单周期不同,在多周期环境下,指令执行被拆分为取指、译码、执行、访存、写回等多个阶段。

在设计中,我重点关注了以下重点:

1. **PC 旧值的维护:**由于 RISC-V 取消了分支延迟槽,在执行跳转或分支指令时,我们需要在阶段二(ID 阶段)之后依然能访问到当前指令的 PC 值(用于计算 $PC + \text{offset}$)。因此,我设计了专门的逻辑来暂存 PC,确保在计算跳转地址时基数正确。
2. **分支提前判定:**为了优化效率,我将 `branch_taken` 的判断逻辑放在执行阶段。一旦判定成功,立即更新 `next_PC`,确保下一个周期能准确进入目标地址。

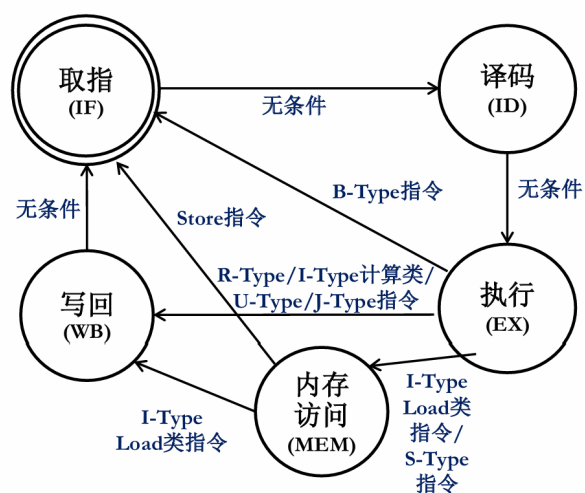


图 11: 多周期处理器的时序图

六、 实验所耗时间

在课后,你花费了大约 5.5 小时完成此次实验。